

# Snort Hands-On Lab Guide

Progressive labs for your Kali (attacker) + Ubuntu/Wazuh (defender) home lab

---

## Lab 0: Installation & Sanity Check

**Goal:** Get Snort installed and confirm it sees live traffic.

### Steps

1. On your Ubuntu box, install prerequisites and Snort:

```
bash

sudo apt update
sudo apt install -y snort
```

During install, you'll be prompted for your network interface and the `HOME_NET` range (e.g. your Ubuntu box's subnet, like `192.168.192.0/24` or whatever `ip a` shows). If unsure, you can set `HOME_NET` to `any` for now and refine later.

2. Confirm the version and config location:

```
bash

snort -V
ls /etc/snort/
```

3. Run Snort in **sniffer mode** to confirm it sees packets:

```
bash

sudo snort -v -i eth0
```

From Kali, ping the Ubuntu box:

```
bash

ping <ubuntu-ip>
```

You should see ICMP packets scroll by in the Snort console. Ctrl+C to stop.

4. Try **packet logger mode**, then inspect what got saved:

```
bash

sudo mkdir -p /var/log/snort
sudo snort -dev -l /var/log/snort -i eth0
```

Generate some traffic from Kali (ping, curl), stop Snort, then read the log:

```
bash

sudo snort -r /var/log/snort/snort.log.<timestamp>
```

**Checkpoint:** You should be comfortable running Snort in sniffer/logger mode and reading back a capture.

---

## Lab 1: Your First Custom Rule — ICMP Detection

**Goal:** Understand rule syntax by writing the simplest possible rule.

### Steps

1. Open (or create) your local rules file:

```
bash

sudo nano /etc/snort/rules/local.rules
```

2. Add this rule:

```
alert icmp any any -> $HOME_NET any (msg:"ICMP Ping Detected"; sid:1000001;
rev:1;)
```

3. Make sure `local.rules` is included in `snort.conf`:

```
bash

grep "local.rules" /etc/snort/snort.conf
```

If not listed, add:

```
include $RULE_PATH/local.rules
```

4. Run Snort in NIDS mode with alerting to console:

```
bash  
  
sudo snort -A console -q -c /etc/snort/snort.conf -i eth0
```

5. From Kali:

```
bash  
  
ping -c 4 <ubuntu-ip>
```

**Expected result:** Snort console prints "ICMP Ping Detected" for each ping.

**Try this too:** Change `(any any -> $HOME_NET any)` to be more specific (e.g., only alert on echo requests using `(itype:8)`), and observe the difference.

---

## Lab 2: Detecting Nmap Port Scans

**Goal:** Detect reconnaissance activity — one of the first things a real attacker does.

### Steps

1. Add a rule for SYN scans:

```
alert tcp any any -> $HOME_NET any (msg:"Possible Nmap SYN Scan"; flags:S;  
threshold:type threshold, track by_src, count 20, seconds 5; sid:1000002; rev:1;)
```

This fires when 20+ SYN packets arrive from the same source within 5 seconds — a strong signal of scanning rather than normal connections.

2. Restart Snort with the updated ruleset:

```
bash  
  
sudo snort -A console -q -c /etc/snort/snort.conf -i eth0
```

3. From Kali, run:

```
bash
```

```
nmap -sS -T4 <ubuntu-ip>
```

4. Observe the alert firing. Then test variations:

- Slow scan: `nmap -sS -T1 <ubuntu-ip>` — does it still trigger? (Probably not, since it stays under your threshold — this teaches you about tuning detection thresholds vs. evasion.)
- Full connect scan: `nmap -sT <ubuntu-ip>` — does your `flags:S` rule still catch it? (It should, since connect scans still start with a SYN.)

5. **Exercise:** Adjust the `count`/`seconds` threshold to catch the slow scan too, without generating excessive false positives on normal traffic. This is the core trade-off in real detection engineering.

---

## Lab 3: Detecting an SSH Brute-Force Attempt

**Goal:** Detect repeated authentication attempts at the network layer (complementing what Wazuh already does at the host log layer).

### Steps

1. Add a rule targeting repeated SSH connection attempts:

```
alert tcp any any -> $HOME_NET 22 (msg:"Possible SSH Brute Force"; flags:S;  
threshold:type threshold, track by_src, count 5, seconds 60; sid:1000003; rev:1;)
```

2. Restart Snort in NIDS mode as before.

3. From Kali, run Hydra against SSH (as in your earlier lab):

```
bash
```

```
hydra -l testuser -P /usr/share/wordlists/rockyou.txt ssh://<ubuntu-ip> -t 4
```

(Use a small custom wordlist if rockyou.txt is too slow for a quick test.)

4. Compare:

- **Wazuh** should alert based on `auth.log` failed password entries (host-based, log content).

- **Snort** should alert based on TCP connection volume (network-based, doesn't need to see login success/failure — just connection attempts).

**Exercise — write this up:** For your notes/portfolio, document the difference: Snort caught the attack pattern before any authentication even occurred (just connection attempts), while Wazuh confirmed based on actual failed logins. This dual-layer detection is exactly why SOCs run both host and network sensors.

---

## Lab 4: Detecting a Specific Exploit Pattern (Payload Matching)

**Goal:** Move beyond traffic-rate rules into content/payload inspection — the technique used for detecting known exploits, malware signatures, and web attacks.

### Steps

1. Set up a simple vulnerable web app on Ubuntu if you haven't already (e.g., DVWA via Docker), or just netcat a fake banner:

```
bash
nc -lvp 8080
```

2. Add a rule that looks for a specific string in traffic, e.g., detecting an SQL injection attempt in an HTTP request:

```
alert tcp any any -> $HOME_NET 80 (msg:"Possible SQL Injection Attempt";
content:"UNION SELECT"; nocase; sid:1000004; rev:1;)
```

`(nocase)` makes the match case-insensitive; `(content)` does a raw substring match.

3. From Kali, send a crafted request:

```
bash
curl "http://<ubuntu-ip>/search?q=test' UNION SELECT username,password FROM user
```

4. Confirm the alert fires.
5. **Exercise:** Try URL-encoding the payload (`(%55NION%20SELECT)`) — does your rule still catch it? This is where you'll discover the value of Snort's HTTP preprocessor (`(http_inspect)`), which normalizes encoded traffic before matching. Look into the `(http_uri)` and `(urilen)` rule options as a next step for handling this properly.

---

## Lab 5: Testing Evasion via Fragmentation

**Goal:** Understand why preprocessors matter, by trying to evade your own rules.

### Steps

1. Keep your Lab 2 (Nmap scan) rule active.
2. From Kali, run a fragmented scan:

```
bash
nmap -f <ubuntu-ip>
```

This splits packets into tiny fragments to try to slip past naive pattern matching.

3. Check whether Snort still detects it. By default, Snort's `frag3` preprocessor reassembles fragments before inspection, so it likely **will** still catch it — that's the point of the exercise.
4. **Exercise:** Look at `/etc/snort/snort.conf` (or `snort.lua` if using Snort 3) and find the `frag3`/stream reassembly settings. Note what they do and why disabling them would reopen this evasion path. Optionally, temporarily disable frag reassembly (in a throwaway config copy, not your main one) and re-test to see evasion succeed — a great "aha" moment for understanding preprocessor importance.

---

## Lab 6: Sending Snort Alerts into Wazuh

**Goal:** Correlate network-based (Snort) and host-based (Wazuh) detections into a single dashboard — the real end goal of this lab series.

### Steps

1. Configure Snort to log alerts in a simple, parseable format. The easiest path is Snort's `fast` alert format to a dedicated log file:

```
output alert_fast: /var/log/snort/alert_fast.log
```

Add this line to your `snort.conf` (or keep the default `-A fast` output if running from CLI).

2. On the Wazuh agent (same Ubuntu box), add the Snort log file as a monitored log source in `ossec.conf`:

xml

```
<localfile>
  <log_format>syslog</log_format>
  <location>/var/log/snort/alert_fast.log</location>
</localfile>
```

3. Restart the Wazuh agent:

bash

```
sudo systemctl restart wazuh-agent
```

4. Write a custom Wazuh decoder to parse Snort's fast-alert format. Create `/var/ossec/etc/decoders/local_decoder.xml` (or add to it):

xml

```
<decoder name="snort-fast">
  <program_name>snort</program_name>
</decoder>
<decoder name="snort-fast-fields">
  <parent>snort-fast</parent>
  <regex offset="after_parent">\[.*\] \[(\d+:\d+:\d+)\] (\S.*) \[.*\].*\{(\S
  <order>id, description, protocol, src_ip, dst_ip</order>
</decoder>
```

(Adjust the regex to match your actual log line format — inspect `alert_fast.log` directly first with `tail -f` while triggering a test alert, and tune the regex to fit.)

5. Write a matching Wazuh rule in `/var/ossec/etc/rules/local_rules.xml`:

xml

```

<group name="snort,">
  <rule id="100010" level="10">
    <decoded_as>snort-fast</decoded_as>
    <description>Snort IDS Alert: $(description)</description>
  </rule>
</group>

```

- Restart the Wazuh manager and agent, then re-run one of your earlier attacks (e.g., the Nmap scan from Lab 2) and confirm the alert now appears **inside the Wazuh dashboard**, not just in Snort's own console/log.

**Checkpoint / milestone:** At this point you have a genuinely SOC-like architecture: network IDS (Snort) feeding a SIEM (Wazuh) that also ingests host logs — the same pattern used in real detection engineering teams.

---

## Lab 7: Capstone — Full Attack Chain Detection Report

**Goal:** Put it all together and document like a real analyst.

### Steps

- From Kali, run a realistic attack chain against Ubuntu in sequence:
  - Nmap scan → SSH brute force (Hydra) → (if you have DVWA/Juice Shop running) a web exploit attempt.
- For each stage, capture:
  - The Snort alert (network layer).
  - The Wazuh alert (host layer, and/or the forwarded Snort alert from Lab 6).
  - Timestamp correlation between the two.
- Write this up as a short incident report:

| Attack Stage | Tool Used | Snort Detection | Wazuh Detection | MITRE ATT&CK Technique |
|--------------|-----------|-----------------|-----------------|------------------------|
| Recon        | nmap      | SID 1000002     | (via Snort fwd) | T1046                  |
| Brute Force  | hydra     | SID 1000003     | auth.log rule   | T1110                  |
| Exploitation | curl/sqli | SID 1000004     | (via Snort fwd) | T1190                  |



This kind of table/report is exactly the artifact worth keeping in a portfolio — it shows detection coverage across the full kill chain, not just isolated tool usage.

---

## Suggested order & pacing

- **Session 1:** Labs 0-2 (install, first rule, scan detection)
  - **Session 2:** Labs 3-4 (brute force, payload matching)
  - **Session 3:** Lab 5 (evasion/preprocessors — conceptually meaty, don't rush it)
  - **Session 4:** Lab 6 (Wazuh integration — the fiddliest one, budget extra time for regex/decoder debugging)
  - **Session 5:** Lab 7 (capstone writeup)
- 

## Reference: Useful Snort commands cheat-sheet

```
bash

snort -V                # version info
snort -v -i eth0        # sniffer mode
snort -dev -l /var/log/snort -i eth0 # packet logger mode
snort -r <logfile>      # read back a saved capture
snort -A console -q -c /etc/snort/snort.conf -i eth0 # NIDS mode, alerts to console
snort -T -c /etc/snort/snort.conf      # test/validate config syntax before running
```

Always run `-T` (test mode) after editing `snort.conf` or rule files — it catches syntax errors before you try to run Snort live.